

SP26 CS570 Week 2 - Lab Notebook Hadoop

January 29, 2026

1 CS570 Week 2 Lab: HDFS & Distributed Thinking

Instructor: Dr. Ragnar Lesch

Duration: 60 minutes

Date: Week 2, Spring 2026

1.1 Instructions

This notebook contains **three parts**:

- **Part 1 (20 min):** HDFS CLI commands
- **Part 2 (40 min):** PySpark coding
- **Part 3:** Reflection questions

How to work through this notebook: 1. Run code cells in order using **Shift+Enter** 2. Answer questions in the markdown cells provided (double-click to edit) 3. Shell commands use **!** prefix — output is captured in the notebook

Prerequisites: - HDFS is running (jps should show NameNode and DataNode) - Python 3.11 and PySpark 4.1.x installed

Submission: Export this notebook as PDF when complete.

2 Part 1: HDFS Command Line Interface

Time: 20 minutes

In this section, you'll use HDFS commands to understand distributed storage concepts.

2.1 Exercise 1.1: Setup Verification (3 minutes)

First, verify your environment is working correctly.

```
[1]: # Check Hadoop installation
!hadoop version
```

Hadoop 3.4.2

Source code repository <https://github.com/apache/hadoop.git> -r

```
84e8b89ee2ebe6923691205b9e171badde7a495c
Compiled by ahmarsu on 2025-08-20T10:30Z
Compiled on platform linux-x86_64
Compiled with protoc 3.23.4
From source with checksum fa94c67d4b4be021b9e9515c9b0f7b6
This command was run using /usr/local/hadoop/share/hadoop/common/hadoop-
common-3.4.2.jar
```

```
[2]: # Check HDFS is running
!hdfs dfs -ls /
```

```
Found 1 items
drwxr-xr-x  - fsehaye supergroup          0 2026-01-21 20:12 /test
```

```
[3]: # Check PySpark
import pyspark
print(f"PySpark version: {pyspark.__version__}")
```

```
PySpark version: 4.1.1
```

2.2 Exercise 1.2: Create and Upload Data to HDFS (5 minutes)

2.2.1 Step 1: Create your HDFS workspace

```
[4]: # Create a directory for this week's work
!hdfs dfs -mkdir -p /user/${whoami}/week2
```

2.2.2 Step 2: Create sample data locally

```
[5]: # Create a local text file with e-commerce transaction data
sample_data = """customer_001,laptop,1299.99,San Francisco,completed
customer_002,headphones,89.99,New York,completed
customer_001,mouse,24.99,San Francisco,completed
customer_003,keyboard,159.99,Los Angeles,pending
customer_002,monitor,449.99,New York,completed"""

with open("sample.txt", "w") as f:
    f.write(sample_data)

print("Created sample.txt")
print("\nContents:")
print(sample_data)
```

```
Created sample.txt
```

```
Contents:
customer_001,laptop,1299.99,San Francisco,completed
customer_002,headphones,89.99,New York,completed
```

```
customer_001,mouse,24.99,San Francisco,completed
customer_003,keyboard,159.99,Los Angeles,pending
customer_002,monitor,449.99,New York,completed
```

2.2.3 Step 3: Upload to HDFS

```
[7]: # Upload file to HDFS
!hdfs dfs -put -f sample.txt /user/$(whoami)/week2/

# Verify upload
!hdfs dfs -ls /user/$(whoami)/week2/
```

Found 1 items

```
-rw-r--r--    1 fsehay supergroup      245 2026-01-22 01:32
/user/fsehay/week2/sample.txt
```

2.2.4 Step 4: Check file metadata

```
[8]: # Check file status and replication
# Format: file_size block_size replication_factor filename
!hdfs dfs -stat "%b %o %r %n" /user/$(whoami)/week2/sample.txt
```

```
245 134217728 1 sample.txt
```

2.2.5 Question 1

What is the replication factor of your file? Why is it set to 1 instead of the default 3?

YOUR ANSWER: - My replication factor is 1 - it's set to 1 because we configured `dfs.replication=1` in `hdfs-site.xml` during setup - Default is 3 in production, but on a single machine with only 1 DataNode, we can only have 1 copy

2.3 Exercise 1.3: Understanding Block Storage (5 minutes)

2.3.1 Step 1: Examine block details

```
[9]: # Check block locations and health
!hdfs fsck /user/$(whoami)/week2/sample.txt -files -blocks -locations
```

```
Connecting to namenode via http://localhost:9870/fsck?ugi=fsehay&files=1&blocks=1&locations=1&path=%2Fuser%2Ffsehay%2Fweek2%2Fsample.txt
FSCK started by fsehay (auth:SIMPLE) from /127.0.0.1 for path
/user/fsehay/week2/sample.txt at Thu Jan 22 05:14:14 PST 2026
```

```
/user/fsehay/week2/sample.txt 245 bytes, replicated: replication=1, 1 block(s):
OK
0. BP-369798605-127.0.1.1-1769054719500:blk_1073741826_1002 len=245 Live_repl=1
```

```
[DatanodeInfoWithStorage[127.0.0.1:9866,DS-8a557173-b0af-4b02-910a-
b6134d7cad13,DISK]]
```

Status: HEALTHY

```
Number of data-nodes: 1
Number of racks:      1
Total dirs:           0
Total symlinks:       0
```

Replicated Blocks:

```
Total size:      245 B
Total files:      1
Total blocks (validated): 1 (avg. block size 245 B)
Minimally replicated blocks: 1 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 0 (0.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 1
Average block replication: 1.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 0 (0.0 %)
Blocks queued for replication: 0
```

Erasure Coded Block Groups:

```
Total size:      0 B
Total files:      0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0
Blocks queued for replication: 0
```

FSCK ended at Thu Jan 22 05:14:14 PST 2026 in 40 milliseconds

The filesystem under path '/user/fsehay/week2/sample.txt' is HEALTHY

2.3.2 Question 2

Answer these based on your fsck output:

- a) How many blocks does your file have?
- b) On which DataNode is the block stored? (IP address and port)
- c) How many racks does your cluster have?
- d) Why do you only see 1 DataNode instead of multiple DataNodes?

YOUR ANSWERS:

- a) 1 block
- b) 127.0.0.1:9866
- c) 1 rack
- d) We're running in pseudo-distributed mode on a single machine.

2.4 Exercise 1.4: Replication in Action (4 minutes)

2.4.1 Step 1: Increase replication factor

```
[10]: # Change replication from 1 to 2
!hdfs dfs -setrep 2 /user/$(whoami)/week2/sample.txt
```

Replication 2 set: /user/fsehayee/week2/sample.txt

2.4.2 Step 2: Verify the change

```
[11]: # Check new replication factor
!hdfs dfs -stat "%r" /user/$(whoami)/week2/sample.txt

# See updated block locations
!hdfs fsck /user/$(whoami)/week2/sample.txt -files -blocks -locations
```

2

Connecting to namenode via <http://localhost:9870/fsck?ugi=fsehayee&files=1&blocks=1&locations=1&path=%2Fuser%2Ffsehayee%2Fweek2%2Fsample.txt>

FSCK started by fsehayee (auth:SIMPLE) from /127.0.0.1 for path /user/fsehayee/week2/sample.txt at Wed Jan 28 01:06:08 PST 2026

/user/fsehayee/week2/sample.txt 245 bytes, replicated: replication=2, 1 block(s):
Under replicated BP-369798605-127.0.1.1-1769054719500:blk_1073741826_1002.
Target Replicas is 2 but found 1 live replica(s), 0 decommissioned replica(s), 0 decommissioning replica(s).

0. BP-369798605-127.0.1.1-1769054719500:blk_1073741826_1002 len=245 Live_repl=1
[DatanodeInfoWithStorage[127.0.0.1:9866,DS-8a557173-b0af-4b02-910a-b6134d7cad13,DISK]]

Status: HEALTHY

```
Number of data-nodes: 1
Number of racks:      1
Total dirs:           0
Total symlinks:       0
```

Replicated Blocks:

```
Total size:    245 B
Total files:    1
Total blocks (validated): 1 (avg. block size 245 B)
Minimally replicated blocks: 1 (100.0 %)
Over-replicated blocks: 0 (0.0 %)
Under-replicated blocks: 1 (100.0 %)
Mis-replicated blocks: 0 (0.0 %)
Default replication factor: 1
Average block replication: 1.0
Missing blocks: 0
Corrupt blocks: 0
Missing replicas: 1 (50.0 %)
Blocks queued for replication: 0
```

Erasure Coded Block Groups:

```
Total size:    0 B
Total files:    0
Total block groups (validated): 0
Minimally erasure-coded block groups: 0
Over-erasure-coded block groups: 0
Under-erasure-coded block groups: 0
Unsatisfactory placement block groups: 0
Average block group size: 0.0
Missing block groups: 0
Corrupt block groups: 0
Missing internal blocks: 0
Blocks queued for replication: 0
```

FSCK ended at Wed Jan 28 01:06:08 PST 2026 in 61 milliseconds

The filesystem under path '/user/fsehayee/week2/sample.txt' is HEALTHY

2.4.3 Question 3

What changed after setting replication to 2?

- a) What replication factor does `stat` show now?
- b) On a single-machine setup, how many actual copies exist? Why?
- c) What would happen on a 3-node cluster with replication set to 2?

YOUR ANSWERS:

- a) 2
 - b) only 1 actual copy exists
 - we only have 1 DataNode running (single machine)
 - HDFS cannot store 2 copies on the same DataNode (defeats the purpose of replication)
 - Replication is for fault tolerance, copies must be on different machines
 - HDFS marks the file as “under-replicated” and waits for another DataNode to join
 - c) HDFS would automatically create 2 copies on 2 different DataNodes.
-

2.5 Exercise 1.5: Data Locality Verification (3 minutes)

2.5.1 Step 1: Download from HDFS

```
[12]: # Download file back to local system
!hdfs dfs -get /user/$(whoami)/week2/sample.txt downloaded_sample.txt
print("Downloaded file from HDFS")
```

Downloaded file from HDFS

2.5.2 Step 2: Compare files

```
[13]: # Compare original and downloaded (no output = identical)
!diff sample.txt downloaded_sample.txt && echo "Files are identical"
```

Files are identical

2.5.3 Question 4

Why does HDFS try to run Map tasks on the same machine that stores the data blocks?

YOUR ANSWER: - To avoid moving large amounts of data over the network. It's faster to move the computation(code) to where the data already is.

2.6 Exercise 1.6: Under the Hood (Optional - 5 minutes)

2.6.1 Part A: See Where HDFS Actually Stores Your Data

HDFS stores blocks as regular files on your local disk. Let's find them:

```
[14]: # Get the configured data directory
!hdfs getconf -confKey dfs.datanode.data.dir
```

file:///tmp/hadoop-fsehay/dfs/data

```
[15]: import subprocess

# Get the data directory path
result = subprocess.run(['hdfs', 'getconf', '-confKey', 'dfs.datanode.data.
↳dir'],
                        capture_output=True, text=True)
data_dir = result.stdout.strip().replace('file://', '')
print(f>Data directory: {data_dir}")

# Find block files
print("\nBlock files found:")
!find {data_dir} -name "blk_*" -type f 2>/dev/null | head -10
```

Data directory: /tmp/hadoop-fsehay/dfs/data

Block files found:

```
/tmp/hadoop-fsehay/dfs/data/current/BP-369798605-127.0.1.1-
1769054719500/current/finalized/subdir0/subdir0/blk_1073741826
/tmp/hadoop-fsehay/dfs/data/current/BP-369798605-127.0.1.1-
1769054719500/current/finalized/subdir0/subdir0/blk_1073741826_1002.meta
```

```
[16]: # Read the actual block file - it contains your data!
import subprocess
result = subprocess.run(['hdfs', 'getconf', '-confKey', 'dfs.datanode.data.
↳dir'],
                        capture_output=True, text=True)
data_dir = result.stdout.strip().replace('file://', '')

print("Contents of a block file:")
!cat {data_dir}/current/BP-*/current/finalized/subdir0/subdir0/blk_1073741*[*!.
↳meta] 2>/dev/null | head
```

Contents of a block file:

```
customer_001,laptop,1299.99,San Francisco,completed
customer_002,headphones,89.99,New York,completed
customer_001,mouse,24.99,San Francisco,completed
customer_003,keyboard,159.99,Los Angeles,pending
customer_002,monitor,449.99,New York,completed
```

2.6.2 Question 5 (Optional)

HDFS shows your file at /user/\$(whoami)/week2/sample.txt, but physically it's stored as block files in /tmp/hadoop-.../dfs/data/.... Why does HDFS use this abstraction instead of just using regular file paths? What benefits does it provide?

YOUR ANSWER:

HDFS uses the block abstraction to provide:

- a. Scalability: Files can be larger than any single disk by splitting across multiple machines
- b. Fault Tolerance - Each block is replicated independently; if one copy is lost, others remain
- c. Parallel Processing - Different blocks can be processed simultaneously on different nodes
- d. Efficient Recovery - Only damaged blocks need re-replication, not entire files
- e. Load Balancing - Blocks distributed evenly across the cluster
- f. Data Locality - NameNode tracks block locations, enabling computation to run where data lives
- g. Simplified Management - Fixed block size makes storage allocation predictable

2.6.3 Part B: HDFS Data Persistence

Run these commands in a separate terminal window, not in the notebook. Stopping HDFS while the notebook is running can cause connection issues.

```
# In a terminal window (not the notebook):

# Stop HDFS services
stop-dfs.sh

# HDFS commands won't work (no NameNode running)
hdfs dfs -ls /user/$(whoami)/week2/
# This will fail with "Connection refused"

# But the actual data files still exist on disk!
find /tmp/hadoop-$(whoami)/dfs/data -name "blk_*" 2>/dev/null | head -3

# Restart HDFS
start-dfs.sh
jps # Verify NameNode and DataNode are running

# Now HDFS commands work again
hdfs dfs -ls /user/$(whoami)/week2/
```

2.6.4 Question 6 (Optional)

In a 3-node cluster with replication=3: - What happens if 1 DataNode stops? Can you still access your files? - What happens if 2 DataNodes stop? Can you still access your files? - What happens if the NameNode stops? What about the data?

Hint: Think about the difference between metadata (file locations) and actual data (blocks).

YOUR ANSWER:

- a. YES. 2 copies still exist. HDFS automatically re-replicates to maintain 3 copies.

- b. YES. 1 copy still exists. File is accessible but at risk. When nodes come back, HDFS re-replicates.
- c. NO. Data blocks still exist physically on DataNode disks, but you cannot access them because:
- NameNode holds the metadata (file → block mapping)
 - Without NameNode, you don't know which blocks make up which file
 - It's like having books in a library but losing the catalog
-

3 Part 2: Max Temperature Problem with PySpark

Time: 40 minutes

Now we'll use Spark to analyze data stored in HDFS.

3.1 Exercise 2.1: Create Weather Dataset

First, we'll create sample weather data and upload it to HDFS.

```
[17]: # Create weather data file locally
weather_data = """San Francisco,2024-01-01,58
San Francisco,2024-01-02,62
San Francisco,2024-01-03,55
New York,2024-01-01,32
New York,2024-01-02,28
New York,2024-01-03,35
Los Angeles,2024-01-01,72
Los Angeles,2024-01-02,75
Los Angeles,2024-01-03,70
San Francisco,2024-01-04,60
New York,2024-01-04,30
Los Angeles,2024-01-04,73"""

# Write to local file
with open('weather_data.txt', 'w') as f:
    f.write(weather_data)

print(" Created weather_data.txt with 12 records")
print("\nSample data:")
print(weather_data[:150] + "...")
```

Created weather_data.txt with 12 records

Sample data:

```
San Francisco,2024-01-01,58
San Francisco,2024-01-02,62
San Francisco,2024-01-03,55
New York,2024-01-01,32
```

New York,2024-01-02,28

New York,2024-01-03,...

```
[18]: # Upload to HDFS
!hdfs dfs -put -f weather_data.txt /user/${whoami}/week2/

# Verify upload
print("HDFS directory contents:")
!hdfs dfs -ls /user/${whoami}/week2/
```

HDFS directory contents:

Found 2 items

```
-rw-r--r--  2 fsehay supergroup      245 2026-01-22 01:32
/user/fsehay/week2/sample.txt
-rw-r--r--  1 fsehay supergroup      307 2026-01-28 17:24
/user/fsehay/week2/weather_data.txt
```

3.2 Exercise 2.2: Initialize Spark and Load Data

Key Concepts: - `SparkSession` - entry point to Spark functionality - `.master("local[*]")` - run locally using all available cores - Reading from HDFS requires the full `hdfs://` URL

```
[1]: from pyspark.sql import SparkSession
import os

# Initialize Spark
spark = SparkSession.builder \
    .appName("MaxTemperature") \
    .master("local[*]") \
    .getOrCreate()

print(" Spark initialized")
print(f" Spark version: {spark.version}")
print(f" App name: {spark.sparkContext.appName}")
```

```
26/01/28 17:36:38 WARN Utils: Your hostname, Fsehay resolves to a loopback
address: 127.0.1.1; using 10.255.255.254 instead (on interface lo)
26/01/28 17:36:38 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another
address
```

Setting default log level to "WARN".

To adjust logging level use `sc.setLogLevel(newLevel)`. For SparkR, use `setLogLevel(newLevel)`.

```
26/01/28 17:36:38 WARN NativeCodeLoader: Unable to load native-hadoop library
for your platform... using builtin-java classes where applicable
```

```
Spark initialized
Spark version: 3.5.3
App name: MaxTemperature
```

```
[2]: # Read CSV file from HDFS
username = os.environ.get('USER', 'your_username')
hdfs_file_path = f"hdfs://localhost:9000/user/{username}/week2/weather_data.txt"

df = spark.read.csv(
    hdfs_file_path,
    header=False,
    inferSchema=True
)

print(" Data loaded from HDFS")
print(f" Number of rows: {df.count()}")
print(f" Number of columns: {len(df.columns)}")
```

```
Data loaded from HDFS
Number of rows: 12
Number of columns: 3
```

```
[3]: # Rename columns for clarity
df = df.toDF("city", "date", "temperature")

print(" Columns renamed")
print("\nDataFrame schema:")
df.printSchema()

print("\nSample data:")
df.show(5)
```

```
Columns renamed
```

```
DataFrame schema:
root
|-- city: string (nullable = true)
|-- date: date (nullable = true)
|-- temperature: integer (nullable = true)
```

```
Sample data:
```

```
+-----+-----+-----+
|      city|      date|temperature|
+-----+-----+-----+
|San Francisco|2024-01-01|      58|
|San Francisco|2024-01-02|      62|
|San Francisco|2024-01-03|      55|
|      New York|2024-01-01|      32|
|      New York|2024-01-02|      28|
+-----+-----+-----+
```

only showing top 5 rows

3.3 Exercise 2.3: Find Maximum Temperature Per City

This is where MapReduce thinking comes in!

Think about it: - **Map:** We already have (city, temperature) pairs in our DataFrame - **Shuffle:** `groupBy("city")` groups all records by city - **Reduce:** `max("temperature")` finds the maximum for each group

```
[4]: from pyspark.sql import functions as F

# Find maximum temperature per city
max_temps = df.groupBy("city").agg(
    F.max("temperature").alias("max_temperature")
)

print(" Calculated maximum temperatures")
print("\nResults:")
max_temps.show()
```

Calculated maximum temperatures

Results:

```
+-----+-----+
|      city|max_temperature|
+-----+-----+
| Los Angeles|          75|
|San Francisco|          62|
|   New York|          35|
+-----+-----+
```

3.3.1 Question 7

In the MapReduce model, what work happens during each phase for the Spark code above?

- a) **Map phase:** What transformation happens to the input data?
- b) **Shuffle phase:** What data movement is required and why?
- c) **Reduce phase:** What aggregation occurs?

YOUR ANSWERS:

- a) Each row of input data is read and the relevant fields are extracted as a key-value pair: (city, temperature). The date column is dropped since it's not needed for finding the maximum

temperature.

b) All temperature values for the same city must be collected together on the same machine. Data is redistributed across the network so that:

- ALL San Francisco temperatures go to one partition
- ALL New York temperatures go to one partition
- ALL Los Angeles temperatures go to one partition This is necessary because temperatures for the same city could be on different nodes.

c) For each group of city temperatures, the `max()` function finds the highest value:

- San Francisco: $\max(58, 62, 55, 60) = 62$
- New York: $\max(32, 28, 35, 30) = 35$
- Los Angeles: $\max(72, 75, 70, 73) = 75$

3.4 Exercise 2.4: Understanding Spark's Execution Plan

Spark has a query optimizer that creates an execution plan.

Use `.explain(True)` to see: - Parsed logical plan (what you wrote) - Analyzed logical plan (with schema info) - Optimized logical plan (Spark's improvements) - Physical plan (actual execution steps)

```
[5]: # Show the execution plan
print("Spark Execution Plan:")
print("=" * 80)
max_temps.explain(True)
```

Spark Execution Plan:

```
=====
== Parsed Logical Plan ==
'Aggregate ['city], ['city, max('temperature) AS max_temperature#58]
+- Project [_c0#17 AS city#31, _c1#18 AS date#32, _c2#19 AS temperature#33]
   +- Relation [_c0#17,_c1#18,_c2#19] csv

== Analyzed Logical Plan ==
city: string, max_temperature: int
Aggregate [city#31], [city#31, max(temperature#33) AS max_temperature#58]
+- Project [_c0#17 AS city#31, _c1#18 AS date#32, _c2#19 AS temperature#33]
   +- Relation [_c0#17,_c1#18,_c2#19] csv

== Optimized Logical Plan ==
Aggregate [city#31], [city#31, max(temperature#33) AS max_temperature#58]
+- Project [_c0#17 AS city#31, _c2#19 AS temperature#33]
   +- Relation [_c0#17,_c1#18,_c2#19] csv

== Physical Plan ==
AdaptiveSparkPlan isFinalPlan=false
```

```

+- HashAggregate(keys=[city#31], functions=[max(temperature#33)],
output=[city#31, max_temperature#58])
  +- Exchange hashpartitioning(city#31, 200), ENSURE_REQUIREMENTS,
[plan_id=122]
    +- HashAggregate(keys=[city#31], functions=[partial_max(temperature#33)],
output=[city#31, max#68])
      +- Project [_c0#17 AS city#31, _c2#19 AS temperature#33]
        +- FileScan csv [_c0#17,_c2#19] Batched: false, DataFilters: [],
Format: CSV, Location: InMemoryFileIndex(1
paths)[hdfs://localhost:9000/user/fsehay/week2/weather_data.txt],
PartitionFilters: [], PushedFilters: [], ReadSchema: struct<_c0:string,_c2:int>

```

3.4.1 Question 8

Look at the physical plan output above and answer:

- How many stages does Spark create?
- Can you identify where the shuffle (Exchange) happens?
- Why does Spark need a shuffle for this operation?

Hint: Look for keywords like “Exchange”, “HashAggregate”, “Scan” in the physical plan.

YOUR ANSWERS:

- 2 stages
 - Stage 1: FileScan → Project → HashAggregate (partial_max) — everything before the shuffle
 - Stage 2: HashAggregate (final max) — everything after the shuffle
 - Yes, this line: Exchange hashpartitioning(city#31, 200), ENSURE_REQUIREMENTS
 - Data for the same city may be on different partitions. To find the max temperature for a city, ALL temperatures for that city must be on the same partition.
-

3.5 Exercise 2.5: Caching for Performance

This is where you’ll see Spark’s advantage over Hadoop!

Experiment: 1. Cache the DataFrame in memory 2. Run the aggregation twice 3. Compare execution times

The difference: - Hadoop writes to disk after every operation - Spark keeps data in RAM between operations

```

[6]: import time

# Cache the DataFrame

```

```

df_cached = df.cache()

# First execution - will load data into memory
print("First execution (loading into cache)...")
start_time = time.time()

max_temps_1 = df_cached.groupBy("city").agg(
    F.max("temperature").alias("max_temperature")
)
max_temps_1.show() # Action triggers execution

first_duration = time.time() - start_time
print(f"\n First execution took: {first_duration:.3f} seconds")

```

First execution (loading into cache)...

```

+-----+-----+
|      city|max_temperature|
+-----+-----+
| Los Angeles|          75|
|San Francisco|          62|
|    New York|          35|
+-----+-----+

```

First execution took: 1.524 seconds

```

[7]: # Second execution - should be faster using cached data
print("Second execution (using cached data)...")
start_time = time.time()

max_temps_2 = df_cached.groupBy("city").agg(
    F.max("temperature").alias("max_temperature")
)
max_temps_2.show() # Action triggers execution

second_duration = time.time() - start_time
print(f"\n Second execution took: {second_duration:.3f} seconds")

# Calculate speedup
if second_duration > 0:
    speedup = first_duration / second_duration
    print(f"\nSpeedup: {speedup:.1f}x faster!")
else:
    print("\n Second execution was essentially instant!")

```

Second execution (using cached data)...

```

+-----+-----+
|      city|max_temperature|

```


Los Angeles	75
San Francisco	62
New York	35

Second execution took: 0.423 seconds

Speedup: 3.6x faster!

3.5.1 Question 9

What happens the second time you run the aggregation on cached data? Why is it faster?

YOUR ANSWER:

The second execution is 3.6x faster because:

- First execution: Spark reads data from HDFS (disk I/O), loads it into memory (RAM), caches it
- Second execution: Data is already cached in RAM. Spark skips the HDFS read entirely and processes the data in memory.

3.6 Exercise 2.6: Challenge - Multi-Metric Analysis (Optional)

Extend your analysis to calculate multiple metrics at once!

Calculate for each city: - Maximum temperature - Minimum temperature - Average temperature
- Count of days recorded

Hint: You can pass multiple aggregation functions to `.agg()`

```
[12]: # TODO: Calculate multiple metrics per city
# Use agg() with multiple functions like:
#   F.max("temperature").alias("max_temp"),
#   F.min("temperature").alias("min_temp"),
#   F.avg("temperature").alias("avg_temp"),
#   F.count("temperature").alias("num_days")

multi_metrics = df.groupBy("city").agg(
    # Add your aggregation functions here
    F.max("temperature").alias("max_temp"),
    F.min("temperature").alias("min_temp"),
    F.avg("temperature").alias("avg_temp"),
    F.count("temperature").alias("num_days")
)
```

```
print("Multi-metric analysis results:")
multi_metrics.show()
```

Multi-metric analysis results:

```
+-----+-----+-----+-----+
|      city|max_temp|min_temp|avg_temp|num_days|
+-----+-----+-----+-----+
| Los Angeles|      75|      70|    72.5|      4|
|San Francisco|      62|      55|   58.75|      4|
|    New York|      35|      28|   31.25|      4|
+-----+-----+-----+-----+
```

3.7 Exercise 2.7: Explore Spark UI (Optional - 5 minutes)

While your SparkSession is running, open <http://localhost:4040> in your browser.

1. Click on the **Jobs** tab - how many jobs did your notebook create?
2. Click on a job, then click on a **Stage** - can you see:
 - Number of tasks?
 - Input/output sizes?
 - Time spent?
3. Click on the **SQL** tab - find your groupBy query and look at the visual DAG.
 - Can you identify the Exchange (shuffle) in the diagram?

This is how you debug and optimize Spark jobs in production!

3.8 Cleanup

Stop Spark and clean up files.

```
[13]: # Stop Spark
spark.stop()
print(" Spark session stopped")
```

Spark session stopped

```
[14]: # Optional: Clean up HDFS and local files
# Uncomment the lines below to run cleanup

# !hdfs dfs -rm -r /user/$(whoami)/week2/

# import os
# for f in ['sample.txt', 'downloaded_sample.txt', 'weather_data.txt']:
#     if os.path.exists(f):
```

```
#         os.remove(f)
#         print(f"Removed {f}")
```

4 Part 3: Reflection Questions

Answer these deeper questions to solidify your understanding of distributed systems concepts.

4.0.1 Question 10: HDFS Replication

- Why does HDFS default to 3x replication in production instead of 2x or 4x?
- In our lab, we used replication=1 for single-machine setup. Why is this acceptable for learning but not for production?
- What are the trade-offs between different replication levels?
- How does replication relate to fault tolerance?

YOUR ANSWER:

HDFS defaults to 3x replication because it provides the optimal balance between fault tolerance and storage cost. With 3 copies, the system can survive 2 simultaneous node failures, and even during the repair process after 1 failure, 2 copies still remain to protect the data. In contrast, 2x replication leaves only 1 copy during repair (risky), while 4x costs 33% more storage for only marginal improvement in reliability.

In our lab, replication=1 is acceptable because we're working with sample data on a single machine that can easily be recreated. In production, data loss is unacceptable — real business data, customer records, and financial information require redundancy. Additionally, our single-machine setup physically cannot replicate since there's only 1 DataNode.

The trade-offs between replication levels involve storage cost, write performance, and fault tolerance. Higher replication provides better fault tolerance and read performance (more copies to serve reads) but costs more storage and slows down writes (each copy must be written). Lower replication saves storage but risks data loss.

Replication directly enables fault tolerance by ensuring data survives hardware failures. HDFS automatically detects failed nodes and re-replicates affected blocks to maintain the target replication level, making the system self-healing.

4.0.2 Question 11: Data Locality

- How does HDFS enable data locality for MapReduce?
- Why is this critical for performance at petabyte scale?
- What would happen if computation always ran on different machines than data?

YOUR ANSWER:

How HDFS enables data locality: HDFS enables data locality through a three-part system. First, the NameNode maintains a metadata map tracking exactly which DataNode stores each block of

every file. Second, when a MapReduce job is submitted, the job scheduler queries the NameNode for block locations. Third, the scheduler assigns Map tasks to the same nodes that store the required data blocks, prioritizing node-local placement, then rack-local, and finally off-rack as a last resort. This way, the small Map code (kilobytes) is sent to the data, rather than moving massive data files across the network.

Why critical at petabyte scale: At petabyte scale, moving data over the network becomes prohibitively expensive. Transferring 1 PB over a 10 Gbps network would take over 9 days, while sending the computation code to 1,000 nodes takes less than a second. Data locality allows thousands of nodes to process their local data in parallel, reducing total processing time from days to minutes. Without it, the network becomes a bottleneck that prevents the system from scaling.

Without data locality: If computation always ran on different machines than data, every operation would require transferring data across the network. The network would become completely saturated, creating a bottleneck that slows down all jobs in the cluster. Processing speed would drop from local disk speed (~500 MB/s) to network speed (~125 MB/s), a 4-10x slowdown. The system would lose its ability to scale — adding more nodes wouldn't help because the network would remain the limiting factor. Resources would be wasted, with data nodes sitting idle while compute nodes wait for data to arrive.

4.0.3 Question 12: MapReduce vs Spark

- When would you choose Hadoop MapReduce over Spark?
- When would you choose Spark over Hadoop?
- What's the fundamental architectural difference?

YOUR ANSWER:

When to choose Hadoop MapReduce over Spark: When the dataset is too large to fit in cluster memory (hundreds of TB), where Spark would spill to disk anyway and lose its in-memory advantage. For simple, one-pass ETL batch jobs where data is read once, transformed, and written — caching provides no benefit. When budget is limited, since Hadoop requires less RAM (cheaper hardware). For very long-running jobs (24+ hours) where disk-based checkpointing provides more reliable fault recovery than Spark's lineage-based recomputation. When maximum fault tolerance is needed, as every MapReduce step is persisted to disk.

When to choose Spark over Hadoop: For iterative algorithms like machine learning, where the same data is processed hundreds of times — Spark's caching makes this 10-100x faster. For interactive data exploration, where a data scientist runs multiple queries on the same dataset and needs fast responses. For complex multi-step pipelines where intermediate results are reused. For real-time stream processing, which Hadoop doesn't support natively. When developer productivity matters, as Spark's Python/SQL APIs are much simpler than writing Java MapReduce code.

Fundamental architectural difference: Hadoop MapReduce is disk-based — it reads from and writes to HDFS (disk) between every processing step. Spark is memory-based — it reads from disk once and keeps intermediate results in RAM for subsequent operations. This means Hadoop pays the cost of slow disk I/O at every stage, while Spark only pays it once. For single-pass jobs the difference is small, but for multi-step or iterative workloads, Spark's in-memory approach is 10-100x faster because RAM access (~10 GB/s) is roughly 50x faster than disk access (~0.2 GB/s).

4.0.4 Question 13: Shuffle Phase

- Why is shuffle the most expensive phase in MapReduce?
- How does Spark optimize shuffle compared to Hadoop?
- What causes a shuffle operation in Spark?

YOUR ANSWER:

Why shuffle is the most expensive phase: Shuffle is the most expensive phase because it involves all the slowest operations simultaneously: serialization (converting objects to bytes), disk I/O (writing sorted data to disk before and after transfer), network transfer (sending data across the cluster), and sorting/merging (combining data from multiple sources). While Map and Reduce phases only require local disk access and CPU processing, shuffle requires coordinated data movement across every node in the cluster. The network becomes a shared bottleneck when all nodes send data simultaneously, and disk I/O at both the sender and receiver sides further compounds the cost.

How Spark optimizes shuffle: Spark optimizes shuffle in several ways compared to Hadoop. First, it performs automatic partial aggregation (combining) before shuffle, dramatically reducing the amount of data transferred — for example, computing a local max per partition before sending just one value per key instead of all values. Second, Spark keeps shuffle data in memory when possible, avoiding Hadoop's mandatory disk writes. Third, Spark's pipelining combines multiple operations into a single stage, reducing the total number of shuffles. Fourth, Adaptive Query Execution (AQE) dynamically adjusts partition sizes during execution to avoid data skew. Fifth, Spark's Tungsten engine uses compact binary serialization that produces much smaller data for transfer compared to Hadoop's verbose Java serialization.

What causes shuffle in Spark: Any operation that requires data from multiple partitions to be combined triggers a shuffle. This includes: `groupBy()` (must collect all values for the same key), `join()` (must match records by key across DataFrames), `orderBy()/sort()` (must compare values across all partitions for global ordering), `distinct()` (must check all partitions for duplicates), and `repartition()` (explicitly redistributes data). Operations like `filter()`, `select()`, `map()`, and `withColumn()` do NOT cause shuffles because each partition can process its data independently without needing data from other partitions.

5 Summary

What you've learned:

1. How to use HDFS CLI commands to interact with distributed storage
2. How blocks and replication work in HDFS
3. How to initialize Spark and read data from HDFS
4. How to perform group-by aggregations (MapReduce-style)
5. How to analyze Spark's execution plans
6. The performance impact of caching (Spark's advantage over Hadoop)

Key Takeaways: - HDFS splits files into 128MB blocks, replicated across nodes for fault tolerance
- Data locality means moving compute to data, not data to compute - Spark's lazy evaluation delays

execution until an action is called - Caching keeps data in RAM between operations (10-100x faster than disk) - GroupBy operations trigger shuffles (expensive network operations)

5.1 Submission

Export this notebook to PDF: - File → Save and Export Notebook As... → PDF - Or: `jupyter nbconvert --to pdf CS570_Week2_Lab.ipynb`

Submit ONE PDF containing all your code outputs and answers to Questions 1-13.

Due: Next class session

Great work!

[]: